

Fiche technique — Contribution du Membre 5

Monte-Carlo, implémentation et expérimentation

Projet : distribution équitable de l'eau sous demande incertaine

Ce document couvre exclusivement la contribution du Membre 5 (M5). Il ne reprend ni ne redérive les parties théoriques complètes de M1, M2, M3 ou M4 ; il n'en présente que ce qui est strictement nécessaire à la compréhension de l'implémentation de M5. Toute information absente des fichiers transmis est signalée explicitement par la mention « *Information non disponible dans les fichiers fournis* » plutôt que supposée ou inventée. Ce document n'est pas le rapport final du projet.

Contents

1	Présentation de la contribution de M5	1
2	Position de M5 dans l'architecture globale	1
3	Objectifs de l'implémentation	2
4	Interfaces avec les autres membres	3
4.1	Dépendances vis-à-vis de M3	3
4.2	Dépendances vis-à-vis de M4	3
4.3	Interfaces avec M6	4
5	Architecture logicielle de ma partie	4
6	Description détaillée des modules	4
6.1	monte_carlo.py	4
6.2	objective.py	5
6.3	gradient_descent.py	6
6.4	run_scenarios.py	7
6.5	Scripts expérimentaux	8
7	Implémentation de la simulation Monte-Carlo	8
8	Implémentation du solveur	9
9	Orchestration des scénarios	10
10	Description des expériences	11
10.1	Expérience 3 — Sensibilité au paramètre de pénalisation μ	11
10.2	Expérience 4 — Vérification de la borne de convergence	12
10.3	Expérience 5 — Maillage du réseau et conditionnement	13
10.4	Expérience 6 — Quartiers à risque	13
11	Résultats obtenus	15
12	Reproductibilité	15

13 Tests et validation de l'implémentation	16
14 Limites et points de vigilance	16
15 Interface avec le Membre 6	17
16 État d'avancement de M5	18
17 Checklist de transmission à M6	19

1 Présentation de la contribution de M5

M5 est responsable du volet « Monte-Carlo, implémentation et expérimentation » du projet. Concrètement, cette responsabilité couvre :

- l'implémentation du simulateur Monte-Carlo de scénarios de demande (`src/probability/monte_carlo.py`) ;
- l'implémentation de la fonction de coût pénalisée et de son gradient (`src/optimization/objective.py`), à partir des dérivations mathématiques fournies par M4 ;
- l'implémentation du solveur de descente de gradient projetée (`src/optimization/gradient_descent.py`), également à partir des dérivations de M4 ;
- l'orchestration de la résolution du problème d'optimisation sur un ensemble de scénarios (`src/simulation/run_scenarios.py`) ;
- la réalisation de plusieurs expériences visant à confronter les résultats théoriques établis par M2 et M4 au comportement observé du code.

M5 implémente ; M5 ne dérive pas la théorie. Les formules utilisées dans `objective.py` et `gradient_descent.py` proviennent d'un document rédigé par M4 (*Aïchatou Traoré*), `docs/section_optimisation` cité nommément dans les deux fichiers. Le rôle de M5 est de traduire ces dérivations en code correct, vérifié, et de produire les expériences qui les confrontent au comportement réel du système.

Un point d'audit important est signalé dès cette section et repris en détail en section 15 : le fichier `exp6_at_risk_districts.py`, bien que présent dans le périmètre nominal de M5 selon la numérotation des six expériences du projet, s'auto-attribue explicitement à M3 dans son propre en-tête et n'utilise aucun des quatre modules de M5. Il est documenté ici tel qu'il a été transmis, avec cette réserve clairement posée.

2 Position de M5 dans l'architecture globale

L'architecture du projet est organisée comme suit :

```

data/
src/graph/           (M1, M2)
src/probability/     (M3 ; monte_carlo.py appartient à M5)
src/optimization/    (M5, à partir des dérivations de M4)
src/simulation/      (M5)
src/evaluation/      (hors périmètre de M5)
experiments/         (scripts 3, 4, 5 attribués à M5 ; script 6 voir section 15)
results/
tests/
report/

```

Le tableau 1 récapitule les étapes du pipeline telles que mentionnées dans les en-têtes des fichiers fournis, avec le responsable associé et le fichier correspondant.

Table 1: Étapes du pipeline mentionnées dans les fichiers fournis.

Étape	Description	Responsable	Fichier
3	Constante de Lipschitz / conditionnement (référéncée, non fournie)	M2	<code>src/graph/graph_analysis.py</code> (non fourni)
4	Modèle de demande $D_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$	M3	<code>demand_model.py</code>
5	Génération de scénarios de demande	M5 (avec M3)	<code>monte_carlo.py</code>
6	Formulation du problème et convexité	M4	<code>objective.py</code> (implémentation M5)
7	Pénalisation de la contrainte d'égalité	M4	<code>objective.py</code> (implémentation M5)
8	Gradient $\nabla J(q)$	M4	<code>objective.py</code> (implémentation M5)
9	Descente de gradient projetée	M5 (à partir de M4)	<code>gradient_descent.py</code>
10	Résolution sur un ensemble de scénarios	M5	<code>run_scenarios.py</code>

3 Objectifs de l'implémentation

D'après les en-têtes des fichiers fournis, l'implémentation de M5 poursuit trois objectifs explicites :

1. Transformer le modèle probabiliste de M3 en scénarios de demande exploitables, avec une mesure explicite de la précision statistique des quantités estimées (voir le docstring de `monte_carlo.py`, qui insiste sur ce point : « *Ce qui est noté, ce n'est pas de savoir tirer des nombres au hasard, c'est de savoir avec quelle précision on connaît les statistiques qu'on en déduit* »).
2. Implémenter fidèlement, sans simplification algébrique intermédiaire, les formules dérivées par M4 pour le coût pénalisé, son gradient et le solveur, de façon à préserver la correspondance entre le rapport et le code (exigence citée dans `gradient.py`¹).
3. Confronter empiriquement deux résultats théoriques du projet au comportement observé du code : la borne de convergence $\eta < 2/L$ (Expérience 4) et le lien entre maillage du réseau et conditionnement (Expérience 5), ainsi qu'analyser la sensibilité du problème au paramètre de pénalisation μ (Expérience 3).

4 Interfaces avec les autres membres

4.1 Dépendances vis-à-vis de M3

M5 dépend du module `src/probability/demand_model.py`, rédigé par M3, via les fonctions suivantes, effectivement importées dans `monte_carlo.py` :

¹Le fichier source la nomme « Contrainte méthodologique 1 ».

- `parametres_demande(reseau)` $\rightarrow (\mu, \sigma)$: vecteurs des moyennes et écarts-types de demande par quartier.
- `matrice_covariance(reseau)` $\rightarrow \Sigma$: matrice de covariance des demandes, construite comme $\Sigma_{ij} = \rho_{ij}\sigma_i\sigma_j$ à partir des corrélations déclarées dans `network_config.json`. Cette fonction lève une exception si Σ n'est pas définie positive.
- `echantillonner(mu, covariance, n_tirages, generateur)` $\rightarrow$ tableau $(n_tirages, p)$: tirages gaussiens multivariés, tronqués à zéro par en bas (`np.clip(..., a_min=0.0, ...)`) pour préserver la positivité physique des débits.
- `estimateur_moyenne`, `estimateur_variance`, `correlations_empiriques` : importées par `monte_carlo.py` mais non appelées dans le corps de ce fichier tel que fourni. Leur usage effectif par M5 n'apparaît pas dans les fichiers transmis.

M5 ne dépend pas de `intervalle_confiance_moyenne` ni de `quartiers_atypiques`, bien que ces deux fonctions existent dans `demand_model.py` et semblent conçues pour le type d'analyse que l'Expérience 6 revendique (voir section 15).

Le contrat d'interface entre `demand_model.py` et `monte_carlo.py` est cohérent : les six fonctions importées existent bien, avec les signatures attendues.

4.2 Dépendances vis-à-vis de M4

M5 dépend d'un document théorique de M4, `docs/section_optimisation_membre4.pdf` (non fourni), pour trois résultats repris littéralement dans le code :

- la formulation pénalisée $J(q) = q^\top Cq + \mu \|Aq - b\|^2$ (section 7 du docstring de `objective.py`) ;
- le gradient $\nabla J(q) = 2Cq + 2\mu A^\top (Aq - b)$ (section 8) ;
- la borne de convergence $\eta < 2/L$ avec $L = \lambda_{\max}(2C + 2\mu A^\top A)$, ainsi que le choix pratique usuel $\eta = 1/L$.

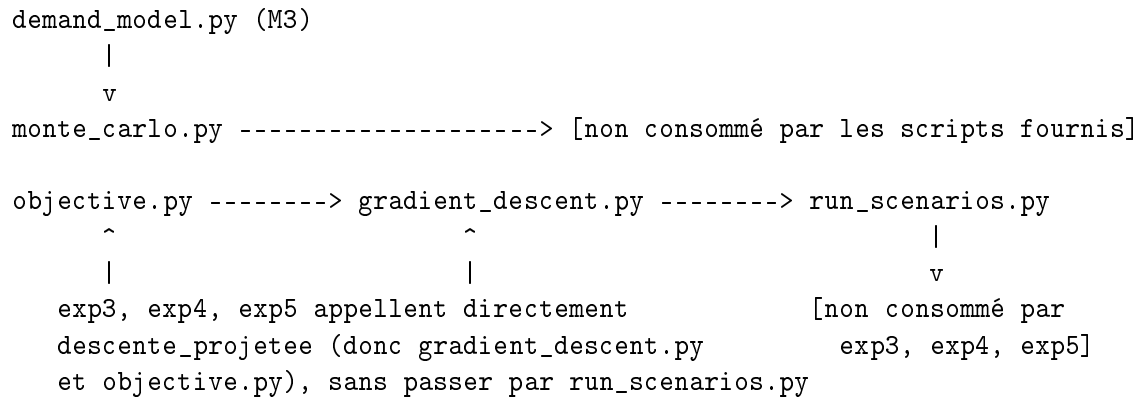
Un écart assumé avec M4 est explicitement documenté dans le code et doit être arbitré par le groupe : le document de M4 (section 5.3, selon le docstring de `gradient_descent.py`) propose trois critères d'arrêt et retient par défaut la norme du gradient $\|\nabla J\| \rightarrow 0$. M5 argumente que ce critère n'est pas adapté à un problème projeté : une composante dont le débit optimal vaut zéro conserve un gradient non nul, que la projection annule à chaque itération, ce qui empêcherait la boucle de s'arrêter avant `max_iterations`. M5 retient donc par défaut le critère de déplacement $\|q_{k+1} - q_k\| < \text{tolerance}$, tout en rendant les trois critères de M4 disponibles via le paramètre `critere` de `descente_projetee`. Ce point est signalé dans le code comme devant être arbitré par M4 pour harmoniser le rapport final.

4.3 Interfaces avec M6

Voir la section 15, dédiée spécifiquement à cette interface.

5 Architecture logicielle de ma partie

Quatre modules constituent l'implémentation de M5, avec les dépendances suivantes entre eux (flèche = « est utilisé par ») :



Ce schéma reflète un constat d'audit à porter à la connaissance du groupe : **aucun des scripts d'expérience fournis (3, 4, 5) n'appelle run_scenarios.py ni monte_carlo.py**. Les trois expériences disponibles résolvent un problème déterministe unique, à la demande moyenne μ obtenue via `parametres_demande`, et non sur un ensemble de scénarios Monte-Carlo. Le module `monte_carlo.py` et les fonctions batch de `run_scenarios.py` (`resoudre_tous`, `descente_projetee_par_lot`, `scenarios_de_stress`) sont implémentés mais non démontrés empiriquement par les fichiers transmis. Ce point est développé en section 14.

6 Description détaillée des modules

6.1 monte_carlo.py

Rôle

Transformer le modèle probabiliste de demande de M3 en scénarios exploitables et en statistiques agrégées.

Objectif

Ne pas se limiter au tirage de nombres aléatoires : fournir une mesure explicite de la précision des statistiques déduites (erreur standard, quantiles).

Entrées

Un objet `reseau` (ou dictionnaire de configuration), un nombre de tirages, une graine optionnelle.

Sorties

Un tableau de scénarios (`n_tirages`, `|quartiers|`), ou un objet `StatistiquesMonteCarlo` (moyenne, variance, erreur standard, quantiles, nombre de tirages).

Principales fonctions

`generer_scenarios`, `agreger`, `erreur_estimation`, `convergence_par_taille`.

Algorithmes

Tirage gaussien multivarié (délégué à `demand_model.echantillonner`) ; calcul de moyenne, variance corrigée (`ddof=1`) et quantiles empiriques via `numpy`.

Dépendances

`src.probability.demand_model` (M3).

Hypothèses

La graine par défaut (`GRAINE_PAR_DEFAULT = 42`) est fixée au niveau du module plutôt qu'à chaque appel, pour qu'un oubli de graine produise un résultat reproductible plutôt qu'un résultat différent à chaque exécution.

Contrôles

`agreger` lève une exception si moins de deux valeurs sont fournies (variance corrigée indéfinie). `convergence_par_taille` lève une exception si une taille demandée dépasse la population disponible, plutôt que de réduire silencieusement la taille.

Fichiers qui l'utilisent

Aucun des fichiers fournis (`exp3`, `exp4`, `exp5`, `exp6`, `run_scenarios.py`) n'importe ce module. Son docstring affirme qu'il alimente `run_scenarios` et les Expériences 2 et 6 ; cette affirmation n'est vérifiée par aucun des fichiers transmis (voir section 15).

6.2 objective.py

Rôle

Implémenter la fonction de coût pénalisée $J(q)$, son gradient, sa hessienne et un contrôle de correction du gradient.

Objectif

Traduire fidèlement, sans simplification algébrique, les formules dérivées par M4.

Entrées

$q \in \mathbb{R}^{|E|}$ (débits), A (matrice d'incidence, forme $(|V|, |E|)$), $b \in \mathbb{R}^{|V|}$ (second membre de conservation), $c \in \mathbb{R}^{|E|}$ (coûts par conduite), $\mu > 0$ (paramètre de pénalisation).

Sorties

Un scalaire ($J(q)$, coût technique, norme de violation) ou un vecteur/matrice (gradient, hessienne).

Principales fonctions

`cout`, `violation_contrainte`, `objectif`, `gradient`, `hessienne`, `verifier_gradient`.

Algorithmes

Calcul direct des formes quadratiques ; différences finies centrées d'ordre $O(h^2)$ pour la vérification numérique du gradient.

Dépendances

Aucune dépendance à un autre module du projet ; utilise uniquement `numpy`.

Hypothèses

Les coûts c_e sont strictement positifs (condition de définie-positivité de $C = \text{diag}(c_e)$, nécessaire à la stricte convexité de J).

Contrôles

`verifier_gradient` compare le gradient analytique aux différences finies centrées ; un écart relatif maximal de l'ordre de 10^{-6} ou moins indique une dérivation correcte. Ce contrôle ne remplace pas la dérivation manuscrite exigée par le sujet, il la vérifie.

Fichiers qui l'utilisent

`gradient_descent.py` (import direct de `gradient`, `hessienne`, `objectif`, `violation_contrainte`) ; `run_scenarios.py` (`cout`, `violation_contrainte`) ; `exp3_sensitivity_mu.py` (`cout`, `hessienne`, `violation_contrainte`) ; `exp5_conditioning_vs_mesh.py` (`hessienne`).

Formulation mathématique implémentée (dérivée par M4, section 6 à 8 de son document) :

$$\text{Problème d'origine : } q^* = \arg \min_q \sum_{e \in E} c_e q_e^2 \quad \text{sous } Aq = b, q \geq 0 \quad (1)$$

$$\text{Pénalisation : } J(q) = q^\top Cq + \mu \|Aq - b\|^2 \quad (2)$$

$$\text{Gradient : } \nabla J(q) = 2Cq + 2\mu A^\top (Aq - b) \quad (3)$$

Le module ne redérive pas la démonstration de convexité de M4 : il en implémente la conséquence directe (voir 6.3 pour la hessienne constante $H = 2C + 2\mu A^\top A$).

6.3 gradient_descent.py

Rôle

Résoudre le problème pénalisé (2) par descente de gradient projetée sur l'orthant positif.

Objectif

Fournir un solveur robuste, avec traçabilité de la convergence et garde-fou contre la divergence numérique.

Entrées

A , b , c , μ , pas η (optionnel), point de départ q_0 (optionnel), tolérance, nombre maximal d'itérations, critère d'arrêt ("déplacement", "gradient" ou "cout").

Sorties

Le couple $(q^*, \text{historique})$, où `HistoriqueConvergence` trace les valeurs de $J(q_k)$, les normes de gradient, les violations de contrainte, le nombre d'itérations, le statut de convergence et le motif d'arrêt.

Principales fonctions

`projeter_orthant_positif`, `pas_maximal_theorique`, `descente_projete`.

Algorithmes

Voir pseudo-code ci-dessous. Projection euclidienne sur $\{q \geq 0\}$ par $P(x) = \max(x, 0)$, appliquée à *chaque* itération (et non une seule fois à la fin, erreur explicitement signalée comme un autre algorithme qui ne converge pas vers le même point).

Dépendances

`objective.py`.

Hypothèses

Si η n'est pas fourni, il est fixé à la moitié de la borne théorique $2/L$, soit $\eta = 1/L$, et ce choix doit être signalé dans le rapport plutôt que présenté comme un réglage manuel. Si q_0 n'est pas fourni, le point de départ est le vecteur nul, admissible pour $q \geq 0$.

Contrôles

Un seuil de divergence (`SEUIL_DIVERGENCE = 1e12`) arrête proprement la boucle si le déplacement devient non fini ou excessif, plutôt que de laisser la sortie se remplir de `inf/nan`. Ce seuil est un garde-fou de mise en œuvre introduit par M5, non issu du document de M4.

Fichiers qui l'utilisent

`run_scenarios.py` (`descente_projete`, `pas_maximal_theorique`); `exp3_sensitivity_mu.py`, `exp4_convergence_check.py`, `exp5_conditioning_vs_mesh.py` (`descente_projete`, `pas_maximal_the`).

Pseudo-code (tel que documenté dans le module) :

Entrées : A, b, c, μ, η, q_0 , tolérance, $k_{\max}$

Pour $k = 0, 1, 2, \dots$:

- $g_k \leftarrow \nabla J(q_k) = 2Cq_k + 2\mu A^\top (Aq_k - b)$
- $q_{k+1} \leftarrow P(q_k - \eta g_k)$, avec $P(x) = \max(x, 0)$
- arrêt si $\|q_{k+1} - q_k\| < \text{tolérance}$ ou $k = k_{\max}$

Sortie : q_k

Deux justifications explicitement portées par le code :

1. $\max(\cdot, 0)$ est la projection euclidienne sur $\{q \geq 0\}$ car le problème $\min_{y \geq 0} \|y - x\|^2$ se sépare composante par composante ; cette séparabilité ne tiendrait pas sur un ensemble couplant les composantes (par exemple des contraintes de capacité $q_e \leq \text{cap}_e$).
2. $\eta < 2/L$ garantit la convergence, avec $L = \lambda_{\max}(2C + 2\mu A^\top A)$: résultat classique de la descente de gradient projetée sur fonction convexe à gradient L -lipschitzien. Cette borne théorique est confrontée à l'expérience dans l'Expérience 4 (section 10.2).

6.4 run_scenarios.py

Rôle

Résoudre le problème d'optimisation sur un ensemble de scénarios de demande, et non uniquement sur la demande moyenne.

Objectif

Industrialiser le test de robustesse de q^* face à des situations variées, exigence explicite du sujet.

Entrées

Un **reseau**, une matrice d'incidence A , un ou plusieurs vecteurs de demande, μ , options du solveur.

Sorties

Un **ResultatScenario** (indice, demandes, q^* , coût technique, violation, nombre d'itérations, convergence) ou une liste de ces objets.

Principales fonctions

`resoudre_scenario`, `resoudre_tous`, `descente_projetee_par_lot`, `scenarios_de_stress`.

Algorithmes

`descente_projetee_par_lot` réplique exactement l'algorithme de `descente_projetee` (même pas, même règle de mise à jour, même projection à chaque itération, même critère de déplacement), mais fait avancer N scénarios simultanément dans des tableaux de forme $(N, |E|)$ au lieu d'une boucle Python scénario par scénario. Le docstring justifie ce choix par un gain de performance (environ 8000 itérations nécessaires à $\mu = 100$ quel que soit le nombre de scénarios ; en boucle, cela représenterait de l'ordre d'un quart d'heure pour 1000 scénarios).

Dépendances

`src.graph.build_graph` (`construire_second_membre`, `vecteur_couts` — M1/M2, non fournis), `src.optimization.gradient_descent`, `src.optimization.objective`, `src.probability.demande` (`parametres_demande`).

Hypothèses

Un scénario ayant atteint son seuil de convergence continue d'être mis à jour jusqu'à la fin de la boucle par lot ; le docstring affirme que cela ne le déplace pas, car il est au point fixe de l'itération projetée.

Contrôles

Le docstring mentionne un test `test_le_lot_donne_le_meme_resultat_que_la_boucle` comparant les deux implémentations (boucle et lot) sur le réseau réel. **Ce fichier de test n'a pas été transmis** ; son existence et son contenu réel restent à vérifier. Information non disponible dans les fichiers fournis.

Fichiers qui l'utilisent

Aucun des scripts d'expérience fournis (`exp3`, `exp4`, `exp5`, `exp6`) n'importe ce module.

6.5 Scripts expérimentaux

Voir section 10 pour le détail complet. Le tableau 2 résume le statut de chaque script fourni.

Table 2: Statut des scripts expérimentaux fournis.

Script	Responsable déclaré	Modules M5 utilisés	Remarque
<code>exp3_sensitivity</code>	<code>M5_mu.py</code> (interprétation M4)	<code>objective.py</code> , <code>gradient_descent.py</code>	Demande moyenne uniquement, pas de Monte-Carlo
<code>exp4_convergence</code>	<code>M5_check.py</code> (interprétation M4)	<code>objective.py</code> (via <code>gradient_descent.py</code>), <code>gradient_descent.py</code>	idem
<code>exp5_conditioning</code>	<code>M5g_vs_mesh.py</code> (interprétation M2)	<code>objective.py</code> , <code>gradient_descent.py</code>	idem
<code>exp6_at_risk_distribution</code>	<code>M5_stats.py</code> l'en-tête du fichier lui-même	Aucun	Voir section 15

7 Implémentation de la simulation Monte-Carlo

Cette section documente le module `monte_carlo.py` tel qu'implémenté. Elle doit être lue avec la réserve posée en section 6.1 : aucun des scripts fournis n'exerce ce module.

Génération des scénarios

`generer_scenarios(reseau, n_tirages, graine=None)` récupère $(\mu, _)$ via `parametres_demande`, la matrice de covariance Σ via `matrice_covariance`, instancie un générateur `numpy` (graine 42 par défaut) et délègue le tirage à `demand_model.echantillonner`.

Utilisation du modèle probabiliste

Le modèle utilisé est entièrement celui de M3 : $D \sim \mathcal{N}(\mu, \Sigma)$, tronqué à zéro par en bas après tirage.

Nombre de scénarios

Non fixé dans le code de M5 : c'est un paramètre (`n_tirages`) laissé à l'appelant. Aucun des fichiers fournis n'appelle `generer_scenarios` avec une valeur concrète. Information

non disponible dans les fichiers fournis quant au nombre de scénarios effectivement retenu pour la suite du projet.

Structure d'un scénario

Une ligne d'un tableau ($n_tirages$, $|quartiers|$), chaque colonne correspondant à un quartier, dans l'ordre retourné par `demand_model`.

Transmission au solveur

Non exercée dans les fichiers fournis : `run_scenarios.resoudre_tous` attend un tableau `scenarios` issu de `generer_scenarios` selon son docstring, mais aucun appel effectif reliant les deux modules n'apparaît dans le code transmis.

Agrégation des résultats

`agregger(valeurs, quantiles=(0.05, 0.25, 0.5, 0.75, 0.95))` calcule moyenne, variance corrigée (`ddof=1`), erreur standard $\hat{\sigma}/\sqrt{N}$ et les quantiles demandés.

Statistiques calculées

Moyenne, variance, erreur standard, quantiles aux niveaux 0,05/0,25/0,5/0,75/0,95 par défaut.

Intervalles de confiance

Non implémentés dans `monte_carlo.py`. Le module calcule une erreur standard, pas un intervalle de confiance au sens statistique (borne de Student ou gaussienne). La fonction `intervalle_confiance_moyenne`, qui calcule un IC de Student à $(n-1)$ degrés de liberté, existe dans `demand_model.py` (M3) mais n'est pas importée par `monte_carlo.py`.

La formule d'erreur standard implémentée,

$$\text{erreur_estimation}(\hat{\sigma}, N) = \frac{\hat{\sigma}}{\sqrt{N}}, \quad (4)$$

est qualifiée dans le code de « volontairement triviale et isolée », le docstring précisant qu'elle est destinée à être confrontée à la décroissance mesurée quand N augmente par l'Expérience 2. **Cette confrontation n'apparaît dans aucun des fichiers fournis pour l'Expérience 2**, qui dépend du travail de M6 selon les informations transmises initialement. Une confrontation empirique de la décroissance en $1/\sqrt{N}$ existe bien dans les fichiers fournis, mais dans `exp6_at_risk_districts.py` (Figure 3, attribuée à M3), pas dans une Expérience 2 attribuée à M5 — voir section 15.

`convergence_par_taille(valeurs, tailles=(100, 1000, 10000))` recalcule les statistiques sur des *préfixes croissants* de la même population de tirages (et non sur des tirages indépendants à chaque taille), choix motivé dans le code par la volonté d'observer une même expérience s'affiner plutôt que de comparer trois expériences différentes.

8 Implémentation du solveur

Fonction objectif

$J(q) = q^T C q + \mu \|Aq - b\|^2$, avec $C = \text{diag}(c_e)$ (équation (2)).

Gradient

$\nabla J(q) = 2Cq + 2\mu A^T(Aq - b)$ (équation (3)), implémenté littéralement d'après la dérivation de M4, sans réécriture algébrique intermédiaire. Une note de performance sans effet sur le résultat est appliquée : le produit est calculé comme $A^T(Aq - b)$ (deux produits matrice-vecteur) plutôt que $(A^T A)q - A^T b$ (qui construirait une matrice $|E| \times |E|$ à chaque appel).

Pénalisation

Le paramètre μ contrôle le compromis entre respect de la conservation des flux et coût technique. La solution pénalisée ne satisfait $Aq = b$ qu'à la limite $\mu \rightarrow \infty$; le résidu $\|Aq^* - b\|$ est une quantité à mesurer et non à supposer nulle.

Descente de gradient

Voir section 6.3 pour le pseudo-code complet.

Projection

Projection euclidienne sur $\{q \geq 0\}$ à chaque itération, $P(x) = \max(x, 0)$.

Critère d'arrêt

Par défaut, critère de déplacement $\|q_{k+1} - q_k\| < \text{tolérance}$ (choix de M5, motivé et documenté en section 4.2). Les critères de norme de gradient et de variation de coût sont également disponibles via le paramètre `critere`.

Gestion des paramètres

η par défaut : $\frac{1}{2} \times \frac{2}{L} = \frac{1}{L}$. q_0 par défaut : vecteur nul. Tolérance par défaut : 10^{-8} .

`max_iterations` par défaut : 10 000 pour `descente_projete`, 100 000 pour `descente_projete_par_lot`

Suivi de la convergence

`HistoriqueConvergence` enregistre, à chaque itération, la valeur de $J(q_k)$, la norme du gradient et la violation de contrainte, pour éviter d'avoir à relancer l'optimisation pour tracer une courbe a posteriori.

Résultats retournés

Le couple $(q^*, \text{historique})$, avec dans l'historique le nombre d'itérations effectuées, un booléen de convergence et le motif d'arrêt (texte libre : critère atteint, divergence détectée, ou nombre maximal d'itérations atteint).

Un garde-fou de divergence (`SEUIL_DIVERGENCE = 1e12`) interrompt la boucle proprement si le déplacement devient non fini ou dépasse ce seuil — utile en particulier pour l'Expérience 4, qui dépasse volontairement la borne théorique $2/L$.

9 Orchestration des scénarios

Le pipeline décrit dans `run_scenarios.py` est le suivant :

demande simulée $\rightarrow$ construction du second membre b (via `construire_second_membre`, M1/M2) $\rightarrow$ optimisation (`descente_projete` ou `descente_projete_par_lot`) $\rightarrow$ solution q^* $\rightarrow$ collecte des résultats (`ResultatScenario`) $\rightarrow$ [agrégation — non implémentée dans ce module] $\rightarrow$ analyse expérimentale.

Deux réserves à porter à la connaissance du groupe :

1. **L'étape d'agrégation n'est pas câblée.** Ni `resoudre_scenario` ni `resoudre_tous` n'appellent `monte_carlo.agreger` sur la liste de résultats produite. La capacité d'agrégation statistique existe (section 7) mais n'est pas reliée à l'orchestration des scénarios dans les fichiers fournis.
2. **Ce module n'est exercé par aucune expérience fournie.** Les fonctions `resoudre_scenario`, `resoudre_tous`, `descente_projete_par_lot` et `scenarios_de_stress` sont implémentées mais non appelées par `exp3`, `exp4`, `exp5` ou `exp6`. Elles sont vraisemblablement destinées aux Expériences 1 et 2, dépendantes du travail de M6 selon les informations transmises.

`scenarios_de_stress(reseau, quantile=0.95)` construit un scénario de demande *extrême marginal* : chaque quartier est individuellement poussé à son propre quantile ($\mu + z_{0.95}\sigma$, composante par composante), ce qui diffère du quantile de la demande totale du réseau (généralement plus faible, car les écarts individuels ne se cumulent jamais parfaitement). Le code signale explicitement que le rapport final doit préciser laquelle des deux lectures est retenue — ce choix n’est pas encore tranché dans les fichiers fournis.

10 Description des expériences

10.1 Expérience 3 — Sensibilité au paramètre de pénalisation μ

Objectif

Mesurer le compromis porté par μ entre respect de la contrainte de conservation et vitesse de convergence.

Question scientifique

Comment le choix de μ affecte-t-il simultanément la violation résiduelle et le nombre d’itérations nécessaires à la convergence ?

Paramètres contrôlés

$\mu \in \{1, 10, 100, 1000\}$; tolérance 10^{-8} ; `max_iterations` = 200 000.

Méthode

Pour chaque μ , résoudre le problème pénalisé à la demande moyenne avec `descente_projete`, en recalculant $\eta = \frac{1}{2} \times \frac{2}{L}$ à chaque valeur de μ (protocole explicitement retenu : le nombre d’itérations mesuré mélange donc l’effet du durcissement du problème et celui du rétrécissement du pas, ce qui est volontaire car les deux effets partagent la même origine algébrique, la matrice $2C + 2\mu A^\top A$).

Protocole expérimental

Réseau synthétique, demande moyenne uniquement (pas de tirage Monte-Carlo).

Métriques

Valeur propre maximale L , conditionnement de H , pas maximal théorique $2/L$, pas utilisé, nombre d’itérations, convergence, coût technique, violation résiduelle.

Sorties produites

Table CSV `results/tables/ exp3_sensibilite_mu.csv` (chemin obtenu via `_bootstrap.TABLEAUX`, module non fourni).

Figures

`results/figures/exp3_sensibilite_mu.png`, deux panneaux : à gauche, $J(q_k)$ en échelle log-log en fonction de k , une courbe par μ ; à droite, violation résiduelle et nombre d’itérations en fonction de μ , en échelle log-log.

Interprétation attendue

La violation résiduelle décroît quand μ augmente, mais la convergence ralentit — lien direct avec la dérivation de l’Étape 7 : augmenter μ resserre la contrainte d’égalité mais alourdit $\lambda_{\max}(H)$, dégrade donc $\kappa(H)$, impose donc un pas plus petit via $\eta < 2/L$, ralentit donc la convergence.

Lien avec la théorie

Satisfait l’exigence d’analyse de sensibilité d’au moins un hyperparamètre (« Contrainte méthodologique 3, exigence (v) », selon le docstring du script).

Limites

Les valeurs numériques réelles (colonnes du CSV, valeurs de L , de $\kappa(H)$, nombre d'itérations effectif) ne figurent pas dans le code source et n'ont pas été transmises. Information non disponible dans les fichiers fournis.

10.2 Expérience 4 — Vérification de la borne de convergence

Objectif

Confronter la borne théorique $\eta < 2/L$ au comportement observé.

Question scientifique

La convergence est-elle stable sous la borne et instable au-delà, comme le prédit la théorie ?

Paramètres contrôlés

$\mu = 100$; facteurs de la borne $\eta/(2/L) \in \{0,1; 0,5; 0,9; 1,1; 1,5\}$; `max_iterations` = 60 000.

Méthode

Pour chaque facteur, calculer $\eta = \text{facteur} \times (2/L)$ et lancer `descente_projetee`, en suivant $J(q_k)$ et la norme du gradient au fil des itérations.

Protocole expérimental

Réseau synthétique, demande moyenne, cinq valeurs de η encadrant franchement le seuil théorique (trois en dessous, deux au-dessus).

Métriques

$J(q_k)$ et $\|\nabla J(q_k)\|$ en fonction de k ; remontée maximale de J entre deux itérations consécutives ; convergence atteinte ou non ; motif d'arrêt.

Sorties produites

Table CSV `results/tables/ exp4_convergence.csv`.

Figures

`results/figures/exp4_convergence.png` : $J(q_k)$ en échelle logarithmique, une courbe par facteur (trait plein sous le seuil, tireté au-dessus), limitée aux 2000 premières itérations affichées.

Interprétation attendue

Convergence stable pour les facteurs inférieurs à 1, oscillation ou divergence au-delà — confirmée ou non par les données numériques réelles, qui n'ont pas été transmises.

Lien avec la théorie

Confronte le résultat classique de convergence de la descente de gradient projetée sur fonction convexe à gradient L -lipschitzien, et la constante de Lipschitz calculée à partir du L fourni par M2 (Étape 3).

Limites

L'erreur à ne pas commettre, explicitement signalée dans le docstring, est de présenter une courbe qui descend comme preuve de convergence : ce que l'expérience démontre est le lien entre la constante de Lipschitz et le comportement de part et d'autre du seuil, pas le simple fait que J décroisse. Les valeurs numériques réelles (borne calculée, nombre d'itérations par facteur) n'ont pas été transmises. Information non disponible dans les fichiers fournis.

10.3 Expérience 5 — Maillage du réseau et conditionnement

Objectif

Vérifier empiriquement le lien entre structure du graphe et stabilité numérique établi à l'Étape 3 (M2).

Question scientifique

Un réseau mal maillé se traduit-il par un conditionnement plus élevé et davantage d'itérations nécessaires à la convergence ?

Paramètres contrôlés

Densités de maillage $\{0; 0,25; 0,5; 0,75; 1\}$; $\mu = 100$; tolérance 10^{-8} ; `max_iterations` = 200 000.

Méthode

Pour chaque variante de maillage (obtenue via `generer_variantes_de_maillage`, M1/M2, non fourni), calculer $\kappa(A)$ et $\kappa(H) = \kappa(2C + 2\mu A^\top A)$, résoudre avec un pas commun à toutes les variantes (protocole retenu explicitement : η fixé à la moitié de la plus petite borne théorique rencontrée parmi toutes les variantes, pour ne pas mélanger l'effet du conditionnement avec celui d'un pas recalculé à chaque variante).

Protocole expérimental

Réseaux synthétiques à densités de connexion croissantes, tous vérifiés connexes.

Métriques

Nombre de conduites, connexité, rang de A , dimension du noyau (nombre de boucles), $\kappa(A)$, $\kappa(H)$, nombre d'itérations, convergence.

Sorties produites

Table CSV `results/tables/ exp5_maillage_conditionnement.csv`.

Figures

`results/figures/exp5_maillage_conditionnement.png`, deux panneaux : à gauche, $\kappa(A)$ et $\kappa(H)$ en fonction du nombre de conduites (axes doubles) ; à droite, nombre d'itérations en fonction de $\kappa(H)$, annoté du nombre de boucles par variante.

Interprétation attendue

Relation croissante (non nécessairement linéaire) entre conditionnement et nombre d'itérations. Le docstring signale explicitement de ne pas sur-interpréter une régression linéaire sur cinq points.

Lien avec la théorie

C'est $\kappa(H)$, et non $\kappa(A)$, qui gouverne la vitesse de la descente de gradient, selon la section 6.3 du document de M4 citée dans le code.

Limites

Conditions de validité rappelées dans le docstring : toutes les variantes doivent rester connexes (sinon le rang de A change de nature et les κ ne sont plus comparables) ; μ , la tolérance doivent rester identiques d'une variante à l'autre. Les valeurs numériques réelles n'ont pas été transmises. Information non disponible dans les fichiers fournis.

10.4 Expérience 6 — Quartiers à risque

Voir la réserve d'attribution détaillée en section 15. Le contenu est documenté ici tel que transmis.

Objectif déclaré (docstring)

Repérer les quartiers dont la demande s'écarte régulièrement des prévisions, et relier ce constat à la robustesse de q^* .

Attribution réelle du code

Le fichier s'auto-attribue explicitement à M3 dans un bloc de commentaires précédant le code exécutable : « Ce script (Étape 4) [...] Implémentation du Membre 3 (Probabilités et Statistiques) ». Le code n'importe que `src.probability.demand_model.DemandModel` ; aucun des quatre modules de M5 n'est utilisé.

Méthode réellement implémentée

Tirage de $N = 10\,000$ scénarios via `DemandModel.sample_demands` (graine 42), calcul des quantiles empiriques à 95 % et 99 %, et du pic maximal observé, par quartier. Des libellés de type de risque (« Topologique (Pont unique) » pour Q1, « Stochastique (Forte variance) » pour Q6, « Transit inter-zones » pour Q4/Q5, « Modéré / Standard » sinon) sont codés en dur dans le script, sans calcul topologique réel à l'appui : le réseau (objet `Reseau/network_config.json` via `charger_reseau`) n'est jamais chargé dans ce script, contrairement aux Expériences 3, 4 et 5.

Écart avec la méthode annoncée

La fonction `quartiers_atypiques` de `demand_model.py`, qui calcule précisément une fréquence de dépassement d'intervalle de confiance par quartier — exactement ce que décrit le docstring de l'Expérience 6 — n'est pas appelée par le script fourni. Le script utilise à la place une analyse par quantiles simples (95 %, 99 %, maximum observé), sans notion de biais volontaire introduit sur des quartiers connus, ni de comparaison au taux de fausses alertes attendu de 5 % mentionné dans le docstring.

Sorties produites

Quatre figures dans `results/figures/` :

1. `fig1_matrices_correlation_covariance.png` : matrices de corrélation R et de covariance Σ du modèle.
2. `fig2_densites_probabilites_demande.png` : densités gaussiennes de demande par quartier.
3. `fig3_convergence_loi_grands_nombres.png` : erreur empirique $\|\hat{\mu}_N - \mu\|_2$ en fonction de $N \in \{10, 25, 50, 100, 250, 500, 1000, 2500, 5000, 10000, 50000\}$, comparée à la pente théorique en $O(1/\sqrt{N})$, en échelle log-log.
4. `fig4_profil_risque_quartiers.png` : boîtes à moustaches de la demande par quartier, avec un seuil de charge critique locale codé en dur à $60 \text{ m}^3/\text{h}$.

Interprétation

Non disponible pour les valeurs numériques ; les figures ne sont pas fournies en tant qu'images dans ce document.

Lien avec la théorie

La Figure 3 réalise concrètement la vérification empirique de la décroissance en $1/\sqrt{N}$ que le docstring de `erreur_estimation` (module de M5) attribue explicitement au rôle de l'Expérience 2. Ce recouvrement est à faire trancher par le groupe : soit l'Expérience 2 (attribuée à M5, dépendante de M6) est redondante avec cette figure, soit elle doit couvrir un aspect différent non encore précisé.

Limites

Le script contient une note d'intégration indiquant qu'un squelette d'origine, qui levait une `NotImplementedError`, faisait planter le script avant d'atteindre le code utile ; les quatre

figures ont donc été produites en appelant `run_experiment_6` directement plutôt que via le point d'entrée `main()` tel qu'exécuté normalement. Le fichier fourni corrige cet ordre, mais ce point historique est à connaître pour la reproductibilité (section 12).

11 Résultats obtenus

Aucun fichier de résultats (CSV de `results/tables/`, images de `results/figures/`) n'a été transmis avec cette demande. Les scripts fournis produisent du code capable de générer ces résultats, mais **aucune valeur numérique réelle** (conditionnement mesuré, nombre d'itérations effectif, violation résiduelle observée, etc.) n'apparaît dans les fichiers source eux-mêmes : ces valeurs ne sont connues qu'à l'exécution.

Information non disponible dans les fichiers fournis pour l'ensemble de cette section. Le tableau 3 récapitule uniquement la nature des résultats que chaque expérience *produirait* à l'exécution, sans préjuger de leur valeur.

Table 3: Nature des résultats attendus par expérience (valeurs non disponibles).

Expérience	Fichier source	Table attendue	Figure attendue
3	<code>results/tables/exp3_sensibilite_mu.csv</code>	$\mu, L, \kappa(H)$, pas, itérations, coût, violation	<code>exp3_sensibilite_mu.png</code>
4	<code>results/tables/exp4_convergence.csv</code>	facteur de la borne, η , convergence, itérations, J final	<code>exp4_convergence.png</code>
5	<code>results/tables/exp5_maillage_conditionnement.csv</code>	densité, conduites, $\kappa(A)$, conditionnements	<code>exp5_maillage_conditionnement.png</code>
6	— (attribué à M3)	non produite (impression console uniquement)	4 figures listées en section 10.4

Pour compléter cette section, transmettre à M5 (ou directement à M6) le contenu réel de `results/tables/` et, si possible, les images de `results/figures/`.

12 Reproductibilité

Environnement

Python avec `numpy`, `scipy` (`scipy.stats`), `matplotlib` (backend `Agg` imposé explicitement dans les scripts d'expérience, pour un rendu sans affichage interactif). Version exacte de Python et des dépendances : information non disponible dans les fichiers fournis (pas de `requirements.txt` ni de fichier d'environnement transmis).

Commandes

Chaque script d'expérience s'exécute indépendamment, via son bloc `if __name__ == "__main__":` qui appelle `main()`. Exemple : `python experiments/exp3_sensitivity_mu.py`. Un module `_bootstrap` est importé en tête de chaque script d'expérience (3, 4, 5) et expose au minimum `CONFIG`, `TABLEAUX` et `FIGURES` ; son contenu réel n'a pas été transmis. Information non disponible dans les fichiers fournis.

Ordre d'exécution

Aucune dépendance d'ordre n'existe entre les Expériences 3, 4 et 5 : chacune recharge le réseau et reconstruit ses propres objets (A , b , coûts) indépendamment. L'Expérience 6, telle que fournie, ne dépend d'aucune des trois autres.

Dépendances

`exp3`, `exp4` et `exp5` dépendent de `data.generate_network.charger_reseau` (M1), `src.graph.build_graph` (M1/M2) et, pour `exp5`, `data.generate_network.generer_variantes_de_maillage` et `src.graph.graph_analysis` (M2). `exp6` dépend uniquement de `src.probability.demand_model.Demand` qui lit `data/network_config.json` (avec un repli sur un chemin relatif `../data/network_config.json` si le premier chemin n'existe pas) et retourne un dictionnaire vide si aucun des deux fichiers n'est trouvé — ce qui ferait alors échouer silencieusement le calcul de μ, σ, Σ dans `parametres_demande/ matrice_covariance` plutôt que de signaler l'absence de fichier.

Fichiers générés

`results/tables/*.csv` et `results/figures/*.png`, selon le détail donné en section 10 et résumé au tableau 3.

13 Tests et validation de l'implémentation

Aucun fichier de test n'a été transmis avec cette demande. Les fichiers source de M5 **réfèrent** l'existence de tests, sans que leur contenu soit disponible :

- `tests/test_projection.py`, mentionné dans le docstring de `projeter_orthant_positif` comme testant cette fonction de manière isolée.
- Un test nommé `test_le_lot_donne_le_meme_resultat_que_la_boucle`, mentionné dans le docstring de `descente_projetee_par_lot` comme comparant les résultats de la version boucle et de la version lot sur le réseau réel.

Le seul contrôle numérique dont le code source complet est disponible est `verifier_gradient` (`objective.py`), qui compare le gradient analytique aux différences finies centrées et retourne l'écart relatif maximal, avec un seuil indicatif de 10^{-6} pour une dérivation jugée correcte. Il s'agit d'une fonction de contrôle, pas d'un test automatisé au sens `pytest` : rien dans les fichiers fournis n'indique qu'elle soit appelée par une suite de tests.

Information non disponible dans les fichiers fournis : contenu réel des tests unitaires, tests d'intégration, couverture de code, cas limites couverts. **À demander explicitement au groupe** pour compléter cette section.

14 Limites et points de vigilance

1. **Le module `monte_carlo.py` n'est exercé par aucune expérience fournie.** Ses fonctions de génération de scénarios et d'agrégation statistique sont implémentées mais non démontrées empiriquement dans les fichiers transmis.
2. **Les fonctions batch de `run_scenarios.py` ne sont exercées par aucune expérience fournie.** `resoudre_tous`, `descente_projetee_par_lot` et `scenarios_de_stress` sont implémentées mais non appelées ailleurs dans les fichiers transmis.
3. **L'étape d'agrégation du pipeline n'est pas câblée** entre `run_scenarios.py` et `monte_carlo.agreger`.
4. **Écart de critère d'arrêt avec M4**, documenté en section 4.2 et non encore arbitré par le groupe.
5. **Attribution de `exp6_at_risk_districts.py` ambiguë**, développée en détail en section 15.
6. **Recouvrement possible entre l'Expérience 2 (attendue de M5/M6) et la Figure 3 de l'Expérience 6 (M3)**, toutes deux portant sur la vérification empirique de la décroissance en $1/\sqrt{N}$.

7. **Paramètre sensible : μ .** L'Expérience 3 documente explicitement que μ contrôle un compromis précision/vitesse ; un choix de μ mal justifié dans le rapport final fragiliserait la comparaison avec la stratégie de référence.
8. **Hypothèse forte : coûts strictement positifs.** La convexité stricte de J dépend de $c_e > 0$ pour tout e ; aucun contrôle explicite de cette hypothèse n'apparaît dans `objective.py` lui-même (elle serait normalement garantie en amont, par M1/M2).
9. **Chargement silencieux d'une configuration manquante.** La classe `DemandModel` de `demand_model.py` retourne un dictionnaire vide si `network_config.json` n'est trouvé à aucun des deux chemins testés, plutôt que de lever une exception explicite — ce qui contraste avec le choix assumé ailleurs dans `demand_model.py` de ne jamais avoir de repli silencieux.
10. **Absence de tests transmis.** Impossible de confirmer l'état réel de validation de `descente_projetee_parallel.py` face à `descente_projetee`, ni de `projeter_orthant_positif`.
11. **Absence de résultats numériques transmis.** Aucune valeur réelle de κ , de nombre d'itérations, de violation résiduelle ou de coût technique n'est disponible pour cette fiche (section 11).

15 Interface avec le Membre 6

Cette section synthétise, à l'intention de M6, l'état exploitable de la contribution de M5.

Modules directement utilisables

`objective.py` et `gradient_descent.py` sont complets, cohérents entre eux et exercés par trois expériences fonctionnelles (3, 4, 5). Ce sont les modules les plus fiables de la contribution de M5 à ce stade de l'audit.

Fonctions constituant des interfaces importantes

`descente_projetee(A, b, couts, mu, ...)` pour résoudre un problème à une demande donnée ; `cout`, `violation_contrainte` pour évaluer une solution ; `pas_maximal_theorique(A, couts, mu)` pour obtenir la borne $2/L$ nécessaire à tout choix de pas.

Données récupérables

Historiques de convergence complets (`HistoriqueConvergence`) pour les Expériences 3, 4, 5 ; résultats structurés par scénario (`ResultatScenario`) si M6 exerce `run_scenarios.py`, ce qui n'a pas encore été fait par les fichiers fournis.

Résultats déjà disponibles

Aucune valeur numérique concrète n'a été transmise avec les fichiers audités (voir section 11). M6 doit obtenir de M5 le contenu réel de `results/tables/*.csv` et `results/figures/*.png` pour intégrer des chiffres au rapport.

Expériences intégrables immédiatement

Les Expériences 3, 4 et 5, sous réserve d'obtenir leurs sorties réelles : le code est complet et cohérent, les protocoles sont documentés et justifiés.

Expériences dépendantes du travail de M6

Les Expériences 1 (comparaison stratégie de référence / solution optimale) et 2 (robustesse Monte-Carlo), selon les informations transmises initialement par M5. Le code fourni suggère que ces deux expériences mobiliseraient `run_scenarios.py` et `monte_carlo.py`, actuellement inutilisés par aucun script fourni.

Statut de l'Expérience 6

À traiter avec prudence : le script est attribué à M3 par son propre en-tête, et non à M5. M6 doit clarifier avec M3 et M5 lequel des deux volets (le script fourni, ou une version utilisant `monte_carlo.py` et `quartiers_atypiques`) doit entrer dans le rapport final, en particulier pour éviter une redondance avec l'Expérience 2.

Fichiers de résultats à consulter

`results/tables/ exp3_sensibilite_mu.csv`, `results/tables/exp4_convergence.csv`, `results/tables/exp5_maillage_conditionnement.csv` ; aucun de ces trois fichiers n'a été transmis à ce stade de l'audit.

Figures intégrables au rapport

`exp3_sensibilite_mu.png`, `exp4_convergence.png`, `exp5_maillage_conditionnement.png`
— sous réserve de vérifier leur contenu réel, non transmis.

Informations méthodologiques devant apparaître dans le rapport final

Le choix du pas $\eta = 1/L$ par défaut et sa justification ; le choix du critère d'arrêt par déplacement (et l'écart assumé avec le document de M4, à faire trancher avant rédaction finale) ; le protocole retenu pour chaque expérience concernant le recalcul ou non de η d'une condition à l'autre (section 10) ; la distinction entre coût technique et terme de pénalisation dans toute comparaison avec la stratégie de référence (`objective.cout` est la métrique pertinente, pas `objective.objectif`).

16 État d'avancement de M5

Table 4: État d'avancement de la contribution de M5.

Élément	État	Fichier	Résultat
Génération de scénarios (<code>generer_scenarios</code>)	Terminé (implémenté)	<code>monte_carlo.py</code>	Non exploité par les scripts fournis
Agrégation statistique (<code>agreger</code> , <code>erreur_estimation</code> , <code>convergence_par_taille</code>)	Terminé (implémenté)	<code>monte_carlo.py</code>	Non exploité empiriquement dans les fichiers fournis
Fonction objectif et gradient	Terminé, avec contrôle numérique	<code>objective.py</code>	Utilisé par exp3, exp4, exp5
Descente de gradient projetée	Terminé	<code>gradient_descent.py</code>	Utilisé par exp3, exp4, exp5
Résolution par lot (<code>resoudre_tous</code> , <code>descente_projetee_par_lot</code>)	Terminé (implémenté)	<code>run_scenarios.py</code>	Non exploité par les scripts fournis
Scénarios de stress (<code>scenarios_de_stress</code>)	Terminé (implémenté)	<code>run_scenarios.py</code>	Non exploité par les scripts fournis
Expérience 3 (sensibilité μ)	Terminé, table et figure produites par le code	<code>exp3_sensitivity_mu.py</code>	Valeurs numériques : à valider (non transmises)

Table 4: État d’avancement de la contribution de M5.

Élément	État	Fichier	Résultat
Expérience 4 (vérification convergence)	Terminé, table et figure produites par le code	<code>exp4_convergence_ch</code>	Valeurs numériques : à valider (non transmises)
Expérience 5 (maillage vs conditionnement)	Terminé, table et figure produites par le code	<code>exp5_conditioning_v</code>	Valeurs numériques : à valider (non transmises)
Expérience 6 (quartiers à risque)	Terminé, 4 figures produites — attribution à confirmer avec le groupe	<code>exp6_at_risk_distri</code>	Attribué à M3 dans le code ; à valider
Expérience 1 (baseline vs optimal)	Dépend d’un autre membre	<code>exp1_baseline_vs_opt</code>	Primaire non disponible
Expérience 2 (robustesse Monte-Carlo)	Dépend d’un autre membre	<code>exp2_monte_carlo_rob</code>	Robustesse non disponible ; recouvrement possible avec la Figure 3 de exp6
Tests unitaires de M5	À valider	référéncés (<code>test_projection.py</code>) et un test de comparaison boucle/lot), non transmis	Information non disponible

17 Checklist de transmission à M6

- ☐ Contenu réel de `results/tables/exp3_sensibilite_mu.csv` transmis.
- ☐ Contenu réel de `results/tables/exp4_convergence.csv` transmis.
- ☐ Contenu réel de `results/tables/exp5_maillage_conditionnement.csv` transmis.
- ☐ Images des figures `exp3_sensibilite_mu.png`, `exp4_convergence.png`, `exp5_maillage_conditionnement.png` transmises.
- ☐ Statut de l’Expérience 6 clarifié avec M3 et le groupe (script conservé tel quel, réattribué, ou remplacé par une version utilisant `monte_carlo.py`).
- ☐ Recouvrement entre l’Expérience 2 et la Figure 3 de l’Expérience 6 tranché.
- ☐ Écart de critère d’arrêt avec le document de M4 (section 4.2) arbitré et harmonisé entre le code et le rapport.
- ☐ Fichiers de tests réels (`tests/test_projection.py` et le test de comparaison boucle/lot) transmis ou leur absence confirmée.

- ☐ Contenu de `network_config.json` transmis, pour documenter précisément le format d'entrée du pipeline.
- ☐ Contenu de `src/graph/build_graph.py` et `src/graph/graph_analysis.py` (M1/M2) obtenu si M6 a besoin de documenter l'interface amont de `run_scenarios.py` et de l'Expérience 5.
- ☐ Choix méthodologique tranché sur la lecture retenue pour `scenarios_de_stress` (quantile marginal par quartier, ou quantile de la demande totale).
- ☐ Décision du groupe sur le nombre de scénarios Monte-Carlo à retenir pour `generer_scenarios`, actuellement non fixé dans le code.